

Midterm
Introduction to Algorithms
601.433/633

Tuesday, October 21st, 3:00pm-4:15pm

Name:

Ethics Statement

I agree to complete this exam without unauthorized assistance from any person, materials, or device.

Signature:

Date:

1 Problem 1: Short Problems (20 points)

For the following, you *do not* need to give proofs or counterexamples. If the problem is multiple-choice, please circle your answer(s).

- (a) For the Union-Find problem, if we use union by rank and path compression, then the rank stored in each node is always the height of this node.

true **false**

- (b) Take the following list of functions and arrange them in ascending order of growth rate. That is, if function $g(n)$ immediately follows $f(n)$ in your list, then it should be the case that $f(n)$ is $O(g(n))$.

$\sqrt{n}$ $\log^* n$ $n^{\log n}$ $2^{\sqrt{\log n}}$

$\log^* n, 2^{\sqrt{\log n}}, \sqrt{n}, n^{\log n}$

- (c) Let $T(n) = 4T(n/7) + 8n$. Then $T(n)$ is asymptotically equal to: $\Theta(n)$

- (d) The worst-case expected running time of Randomized Quicksort is: $\Theta(n \log n)$

- (e) Every leaf of a red-black tree has the same depth.

true **false**

- (f) The time to do a lookup in a B-tree is independent of the parameter t used for the tree.

true false

- (g) There is a heap data structure which has $O(1)$ amortized Insert and $O(1)$ amortized Extract-Min

true **false**

- (h) Let H be a universal family of hash functions from a universe U onto a table of size M . Then $\Pr_{h \sim H}[h(e) = i] = 1/M$ for all elements $e \in U$ and table positions $i \in \{1, 2, \dots, M\}$.

true **false**

- (i) Least-significant first radix sort is correct no matter which sorting algorithm we use on each digit.

true **false**

- (j) There is an algorithm which can find the $n^{1/10}$ -smallest number in an unsorted array of n numbers in $O(n)$ time.

true false

2 Problem 2: Searching / Sorting (20 points)

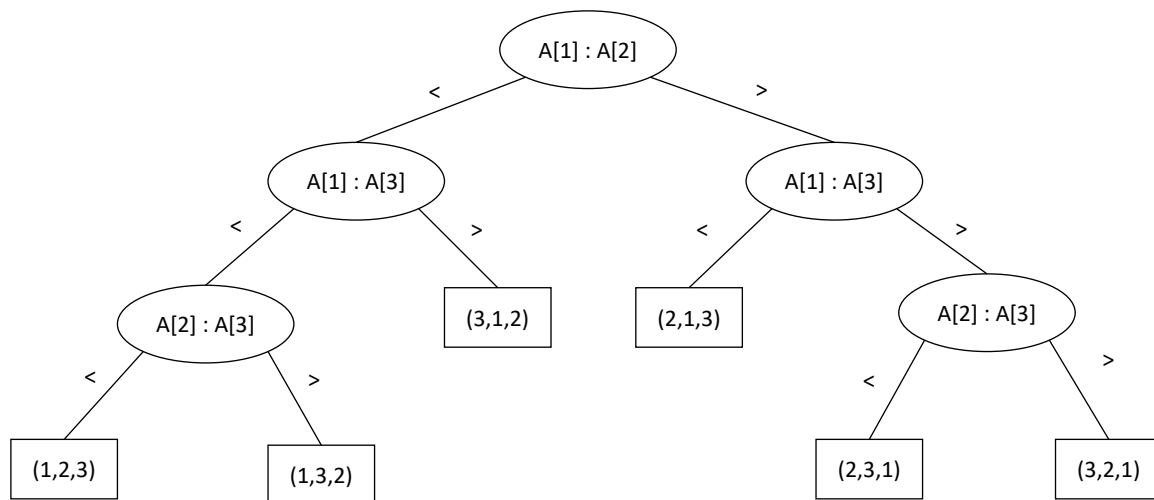
- (a) (10 points) Suppose that we modified the BPFRT (linear-time selection) algorithm so that it finds the median-of-medians of groups of size 15, rather than groups of size 5 as shown in class. What is the recurrence relation that characterizes its running time? Recall that for the normal BPFRT algorithm (groups of size 5), the recurrence relation for the time $T(n)$ was $T(n) \leq T(n/5) + T(7n/10) + cn$. Explain your answer, but you do not need to prove it.

Solution. There will be $n/15$ groups (and so $n/15$) medians, and thus the recursive call on the medians will take time $T(n/15)$. In half of these groups ($n/30$) the median of the group will be larger than the median of the medians and so at least 8 elements of the group will be larger than the median of the medians, and hence $|G| \geq 8(n/30)$, which implies that $|L| \leq 22n/30 = 11n/15$. By the same logic applies to the groups whose median is less than the median of the medians, we know that $|G| \leq 22n/30 = 11n/15$. Hence the recursive call on L or G takes time at most $T(11n/15)$. As with normal BPFRT, computing the median of each group takes time $O(1)$, so dividing into groups and computing the median of each of them takes time $O(n)$. Splitting the array based on the median of medians also takes time $O(n)$. Hence the total running time is at most

$$T(n) \leq T(n/15) + T(11n/15) + cn$$

- (b) (10 points) We talked in class about the correspondence between sorting algorithms and decision trees. Consider deterministic quicksort where the pivot is always the first element. Draw the decision tree corresponding to this algorithm on an array A of size 3 with entries $[A[1], A[2], A[3]]$.

Solution.



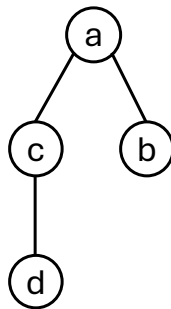
3 Problem 3: Data Structures (20 points)

- (a) (10 points) This problem is about the Union-Find algorithm which we analyzed to show that the amortized cost of an operation is $O(\log^* n)$. Suppose we do the following operations in order, where ties in rank of the Link operation are broken by increasing the rank of the first argument (as in class).

- Make-Set(a)
- Make-Set(b)
- Make-Set(c)
- Make-Set(d)
- Union(a,b)
- Union(c,d)
- Union(b,d)

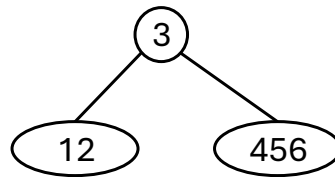
Draw the resulting forest.

Solution.



- (b) (10 points) Suppose that we insert the keys 1,2,3,4,5,6 (in that order) into a B-tree with parameter $t = 3$ using the insertion algorithm from class. Draw the resulting tree.

Solution.



4 Problem 4: Amortized Analysis (20 points)

Recall that a stack is a last-in first-out data structure with the following two operations:

- (1) $\text{Push}(x)$: pushes an element x onto the stack
- (2) $\text{Pop}()$: returns the element in the stack that was most recently added to the stack, and removes this element from the stack. You may assume that $\text{Pop}()$ is never called on an empty stack.

Suppose that we have an implementation of a stack in which $\text{Push}(x)$ has cost 2 and $\text{Pop}()$ has cost 8.

- (a) (10 points) What is the smallest value of k for which the amortized cost of both operations is at most k ? Prove that both operations have amortized cost at most k through either a banking argument or a potential argument.

Solution: $k = 5$. Consider the potential function $\Phi = 3 \times \text{number of elements in the stack}$. Then Φ is initially 0 and is obviously nonnegative, so is a valid potential function. The amortized cost of Push is $2 + \Delta\Phi = 5$, and the amortized cost of Pop is $8 + \Delta\Phi = 8 - 3 = 5$

- (b) (10 points) Now suppose that the costs are reversed: $\text{Push}(x)$ has cost 8 and $\text{Pop}()$ has cost 2. What is the smallest value of k for which the amortized complexity of the both operations is at most k ? Justify your answer, but formal proof is not necessary.

Solution: $k = 8$. Consider a sequence of n operations in which every operation is a Push. Then the total cost is $8n$, so the amortized cost of Push cannot be below 8.

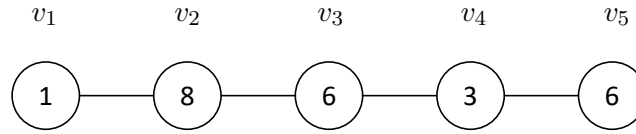
5 Problem 4: Dynamic Programming (20 points)

Let $P = (V, E)$ be the path graph: $V = \{v_1, v_2, v_3, \dots, v_n\}$ and $E = \{\{v_i, v_{i+1}\} : 1 \leq i \leq n-1\}$. Let $w : V \rightarrow \mathbb{R}_{\geq 0}$ be a nonnegative weighting of the vertices.

We say that a set $U \subseteq V$ is *independent* if $\{x, y\} \notin E$ for all $x, y \in U$. In other words, a set of vertices is independent if no two are adjacent. Our goal will be to find the value of the maximum weight independent set in the path graph, i.e., the value

$$\max_{U \subseteq V : U \text{ independent}} \left(\sum_{u \in U} w(u) \right).$$

For example, consider the five node path graph with the following weights:



In this example, the maximum weight independent set consists of $\{v_2, v_5\}$ and has weight $8+6 = 14$.

- (a) (10 points) Let $OPT(i)$ denote the total weight of the maximum weight independent set of the prefix ending at i , i.e., of the subgraph $\{v_1, v_2, \dots, v_i\}$. Write a formula for $OPT(i)$ in terms of the smaller subproblems. Justify your answer (formal proof not necessary).

Solution. Consider $OPT(i)$. If v_i is part of this optimal solution, then v_{i-1} cannot be but we can choose the optimal solution for the prefix ending at $i-2$. So in this case, $OPT(i) = w(i) + OPT(i-2)$. On the other hand, if v_i is not part of this optimal solution, then the optimal solution is the same as for the prefix ending at $i-1$, so $OPT(i) = OPT(i-1)$. Thus we get that

$$OPT(i) = \begin{cases} 0 & \text{if } i = 0 \text{ or } i = -1 \\ \min(w(i) + OPT(i-2), OPT(i-1)) & \text{otherwise} \end{cases}$$

- (b) (10 points) Now consider the obvious dynamic programming algorithm (top-down or bottom-up) based on your answer to part (b). What is its running time? Justify your answer (formal proof not necessary).

Solution. Each table entry takes $O(1)$ time to fill in (two table lookups, a sum, and a min), and there are $O(n)$ table entries. Thus the running time is $O(n)$.

